

CSE 3150 - C++ Essentials

Programming Assignment 6: Page Memory Management

In this assignment, you are to implement the page replacement algorithms for managing virtual memory in a modern operating system. In a modern computer, the virtual memory is organized into pages (or block of memory). We use an integer to represent a page. That is, each page has a distinct integer as its ID. The main memory has a limited capacity and can only hold up to K pages. Here, K is the capacity of the main memory. An important problem regarding to virtual memory is the page replacement: when the memory reaches its limit (i.e., having K pages) and a new page (not currently in memory) is to be add, then need to swap out one current page to make room for this new page. Virtual memory is an important subject in computer science. If you haven't learned about it, it is time to get to know it (Here gives an overview of page replacement algorithms: https://en.wikipedia.org/wiki/Cache_replacement_policies). Or check leetcode 146. LRU Cache.

There are many different page replacement algorithms. In this programming assignment, you need to implement a class that supports following methods. These methods will be used to implement the Least Recently Used (LRU) algorithm and First-In-First-Out (FIFO) algorithm which will be tested in the provided test code. Briefly, LRU policy is to evict the page that is least recently used, while FIFO policy evicts the oldest page in memory (regardless of usage pattern).

This class would support the following algorithms:

1. *Constructor*: take the number of pages (capacity of main memory). This would initialize the memory.
2. *AccessPage*: this function accesses a specific page; you should run LRU upon this request to access page.
3. *RunOpt*: run the optimal page replacement on all the pages that have been accessed so far. Return the number of page faults by this OPT algorithm. See the optimal page replacement algorithm in the link above.
4. *RunFIFO*: run the FIFO (First-In-First-Out) page replacement on all the pages that have been accessed so far. Return the number of page faults by this FIFO algorithm. FIFO evicts the oldest page (first one loaded into memory), regardless of access pattern.
5. *GetNumPageFaults*: return the number of page faults so far (by LRU).

The following lists things to note when you implement this class.

1. You should define the data structure of the virtual memory as data member in the class.
2. Use STL set/vector/map when applicable. These STL utilities have useful methods/functions that can make the implementation easier. You don't need to worry about the time complexity.
3. Read the code given in the test code to understand how the test code works. This can help you understand more deeply what the functions are meant to do. In particular, pay attention to the following three main test functions:
 - (a) *RunLRU*: it takes a list of pages to request (the pointer to the array of pages and the size of array), the size (number of pages) of the memory (memoryCapacity, and the the number of page faults it will be when you run the LRU algorithm. If the number of faults from your implementation doesn't match, it will throw an exception.

- (b) *RunOpt*: the same inputs. The only difference is, it runs the optimal algorithm and examines whether the numbers of page faults match. See the optimal page replacement algorithm in the link above.
- (c) *RunFIFO*: the same inputs. It runs the FIFO algorithm and examines whether the numbers of page faults match. FIFO is simpler than LRU as it only tracks insertion order, not access history.

Example: Test Case 1 Walkthrough

To help you understand how different page replacement algorithms work, let's trace through the first test case step-by-step.

Test Case Parameters

- **Page Access Sequence:** {1, 2, 3, 4, 5, 1, 3, 1, 6, 3, 2, 3}
- **Memory Capacity:** 4 pages
- **Expected Results:**
 - LRU: 8 page faults
 - OPT: 6 page faults
 - FIFO: 9 page faults

LRU (Least Recently Used) - 8 Page Faults

LRU evicts the page that has not been accessed for the longest time. When a page is accessed, it becomes the “most recently used.”

Step	Access	Memory State	Action	Fault?	Total
1	1	[1]	Load page 1	Yes	1
2	2	[2, 1]	Load page 2	Yes	2
3	3	[3, 2, 1]	Load page 3	Yes	3
4	4	[4, 3, 2, 1]	Load page 4 (full)	Yes	4
5	5	[5, 4, 3, 2]	Evict 1 (LRU), load 5	Yes	5
6	1	[1, 5, 4, 3]	Evict 2 (LRU), load 1	Yes	6
7	3	[3, 1, 5, 4]	Hit! Move 3 to front	No	6
8	1	[1, 3, 5, 4]	Hit! Move 1 to front	No	6
9	6	[6, 1, 3, 5]	Evict 4 (LRU), load 6	Yes	7
10	3	[3, 6, 1, 5]	Hit! Move 3 to front	No	7
11	2	[2, 3, 6, 1]	Evict 5 (LRU), load 2	Yes	8
12	3	[3, 2, 6, 1]	Hit! Move 3 to front	No	8

OPT (Optimal) - 6 Page Faults

OPT evicts the page that will not be used for the longest time in the future (requires knowing all future accesses).

Step	Access	Memory State	Action	Fault?	Total
1	1	[1]	Load page 1	Yes	1
2	2	[1, 2]	Load page 2	Yes	2
3	3	[1, 2, 3]	Load page 3	Yes	3
4	4	[1, 2, 3, 4]	Load page 4	Yes	4
5	5	[1, 2, 3, 5]	Evict 4 (never used)	Yes	5
6	1	[1, 2, 3, 5]	Hit!	No	5
7	3	[1, 2, 3, 5]	Hit!	No	5
8	1	[1, 2, 3, 5]	Hit!	No	5
9	6	[2, 3, 5, 6]	Evict 1 (not used again)	Yes	6
10	3	[2, 3, 5, 6]	Hit!	No	6
11	2	[2, 3, 5, 6]	Hit!	No	6
12	3	[2, 3, 5, 6]	Hit!	No	6

Key Insight: OPT knows page 4 will never be used again (step 5), so it evicts 4 instead of keeping it. This saves 2 page faults compared to LRU.

FIFO (First-In-First-Out) - 9 Page Faults

FIFO evicts the oldest page in memory (first one loaded), regardless of how recently it was accessed.

Step	Access	Memory State	Action	Fault?	Total
1	1	[1]	Load page 1	Yes	1
2	2	[1, 2]	Load page 2	Yes	2
3	3	[1, 2, 3]	Load page 3	Yes	3
4	4	[1, 2, 3, 4]	Load page 4 (full)	Yes	4
5	5	[2, 3, 4, 5]	Evict 1 (oldest)	Yes	5
6	1	[3, 4, 5, 1]	Evict 2 (oldest)	Yes	6
7	3	[3, 4, 5, 1]	Hit!	No	6
8	1	[3, 4, 5, 1]	Hit!	No	6
9	6	[4, 5, 1, 6]	Evict 3 (oldest)	Yes	7
10	3	[5, 1, 6, 3]	Evict 4 (oldest)	Yes	8
11	2	[1, 6, 3, 2]	Evict 5 (oldest)	Yes	9
12	3	[1, 6, 3, 2]	Hit!	No	9

Key Insight: FIFO does not consider access patterns. At step 10, it evicts page 3 even though it was just accessed at step 7, causing an immediate fault when page 3 is requested again at step 10.